

Algoritmos Bioinspirados para Otimização

Aluno: Leonardo Ronald Perin Rauta

Professor: Carlos Alberto Bavastri

1. Introdução

No campo da Inteligência Artificial são desenvolvidos muitos algoritmos inspirados na natureza para tentar solucionar problemas complexos. Essas técnicas são chamadas de algoritmos bioinspirados, ou técnicas bioinspiradas. Conforme consolidado por Russell e Norvig (2013) na literatura clássica de IA, a construção de agentes racionais muitas vezes esbarra em ambientes com espaços de estados gigantescos, estocásticos e matematicamente intratáveis, onde métodos determinísticos tradicionais falham devido ao custo computacional, conhecidos como problemas NP-difíceis.

Para contornar essas barreiras, várias técnicas bioinspiradas foram desenvolvidas. Enquanto algumas vertentes focam no aprendizado, como as redes neurais artificiais que se inspiram no funcionamento do cérebro humano, ou os algoritmos genéticos inspirados na teoria da evolução, outras foram aplicadas e adaptadas para a otimização matemática, buscando máximos e mínimos globais em superfícies não-lineares.

Além dos algoritmos genéticos e das redes neurais artificiais, várias outras técnicas foram desenvolvidas na subárea da otimização heurística chamada inteligência de enxame (*Swarm Intelligence*). Estes métodos modelam o comportamento coletivo e descentralizado de sistemas naturais, como colônias de formigas, enxames de abelha, bando de pássaros, entre outros. A premissa fundamental na IA de enxame é a emergência, em que agentes extremamente simples, seguindo regras probabilísticas locais, geram um comportamento global inteligente e capaz de resolver problemas complexos de otimização multivariável. A seguir serão apresentadas algumas técnicas apresentadas por Russel e Norvig (2013).

2. Otimização por Enxame de Partículas (*Particle Swarm Optimization - PSO*)

Proposto originalmente por Kennedy e Eberhart em 1995, o enxame de partículas é um algoritmo heurístico de otimização contínua inspirado na coreografia e no fluxo de informação de um bando de pássaros voando em busca de alimento.

2.1. Funcionamento do algoritmo

No enxame de partículas, a população é chamada de enxame e cada indivíduo é uma partícula. Em vez de operadores genéticos (*crossover*/mutação), as partículas navegam pelo espaço de busca ajustando fisicamente suas trajetórias. Cada partícula i possui uma posição atual x_i e uma velocidade vetorial v_i .

A cada iteração, a partícula atualiza sua velocidade se baseando em três componentes matemáticas, e seguindo a equação 1.

$$v_i^{t+1} = w \cdot v_i^t + c_1 r_1 (P_{best,i} - x_i^t) + c_2 r_2 (G_{best} - x_i^t) \quad (1)$$

1. Inércia ($w \cdot v_i^t$): Mantém a partícula na direção que ela já estava indo, promovendo a exploração do espaço.

2. Componente Cognitiva ($c_1 r_1$): Atração para a melhor posição que a própria partícula já encontrou na história (P_{best}). Representa a memória individual.

3. Componente Social ($c_2 r_2$): Atração para a melhor posição encontrada por qualquer partícula de todo o enxame (G_{best}). Representa a comunicação e inteligência coletiva.

Onde c_1 e c_2 são constantes de aceleração e r_1 , r_2 são variáveis aleatórias uniformes.

Em seguida, a posição é atualizada: $x_i^{t+1} = x_i^t + v_i^{t+1}$. A convergência ocorre de forma contínua e fluida.

2.2. Algoritmo no MATLAB

O algoritmo de enxame de partículas é implementado no MATLAB no *Global Optimization Toolbox* através da função `particleswarm`. O solver opera inteiramente com vetores em ponto flutuante e ajusta dinamicamente a inércia ao longo das gerações para refinar a busca, sendo amplamente utilizado para calibragem contínua devido à sua extrema velocidade vetorial.

3. Otimização por Colônia de Formigas (*Ant Colony Optimization - ACO*)

Introduzido por Marco Dorigo em 1992, a colônia de formigas resolve problemas da mesma forma como formigas reais encontram o caminho mais curto entre o ninho e uma fonte de alimento utilizando comunicação química.

3.1. Funcionamento do algoritmo

A colônia de formigas é baseada em grafos, em que formigas artificiais caminham pelos nós do grafo construindo soluções passo a passo. A escolha do próximo nó j a

partir de um nó i não é determinística, mas probabilística, definida pela regra de transição, equação 2.

$$P_{ij} = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum (\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta} \quad (2)$$

τ_{ij} representa a quantidade de feromônio virtual depositada na aresta.

η_{ij} é a informação heurística (geralmente o inverso da distância, $1/d_{ij}$).

α e β são pesos que regulam a importância do feromônio versus a heurística.

As formigas depositam feromônio nos caminhos que percorrem. Caminhos mais curtos, que rendem um *fitness* melhor, recebem feromônio mais rápido e em maior quantidade. Para evitar que o algoritmo fique preso em mínimos locais, é aplicada uma evaporação de feromônios. A cada iteração, os feromônios antigos decaem multiplicados por uma taxa $(1-\rho)$. Eventualmente, o enxame converge para a trilha mais forte.

3.2. Algoritmo no MATLAB

O MATLAB não possui uma função nativa para o algoritmo de colônia de formigas.

4. Algoritmo da Colônia de Abelhas Artificiais (*Artificial Bee Colony - ABC*)

Desenvolvido por Dervis Karaboga em 2005, o algoritmo simula o comportamento de busca por néctar e o compartilhamento de informações em uma colmeia de abelhas.

4.1. Funcionamento do algoritmo

Da mesma forma que uma colônia de abelhas, a população é dividida em três grupos de agentes com tarefas distintas.

Operárias: Exploram fontes de alimento conhecidas. Elas geram uma solução vizinha variando uma coordenada aleatória através da equação 3.

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj}) \quad (3)$$

(Onde x_k é um vizinho aleatório e ϕ um número aleatório entre -1 e 1).

Observadoras: Aguardam na colmeia pelas operárias, que realizam a dança compartilhando a quantidade de néctar. As observadoras escolhem quais fontes explorar usando o método de roleta clássica $P_i = \frac{fit_i}{\sum fit_n}$.

Exploradoras: A colônia de abelhas possui um mecanismo contra mínimos locais chamado *limit*. Se uma fonte de comida não melhora após *limit* tentativas, ela é declarada esgotada. A operária descarta essa solução, se torna uma exploradora, e gera um ponto completamente aleatório no espaço de busca.

4.2. Algoritmo no MATLAB

O MATLAB não possui uma função nativa para o algoritmo de colmeia de abelhas.

5. Conclusão

As técnicas puramente matemáticas vistas em sala de aula se referem à abordagens determinísticas e baseadas em derivadas, possuindo limitações significativas em problemas do mundo real, onde a topologia da função objetivo é ruidosa, não-convexa ou combinatória. Conforme a literatura de inteligência artificial, nesses cenários as técnicas heurísticas se tornam não apenas opções, mas necessidades computacionais.

As técnicas de inteligência de enxame, como os algoritmos aqui apresentados, provam que o design de sistemas multiagentes com cooperação descentralizada é uma ferramenta útil em muitos casos de otimização. A grande força dessas técnicas bioinspiradas está no balanceamento dinâmico entre a forma de realizar a busca global por novas áreas, e no refinamento das melhores soluções, a depender do algoritmo a ser utilizado.

Como o MATLAB já possui implementação da técnica de enxame de partículas (`particleswarm`), é reforçada a validade industrial dessas técnicas bioinspiradas desenvolvidas, demonstrando que a simulação matemática do comportamento natural é uma estratégia robusta e indispensável para a otimização na engenharia.

6. Referências

MATHWORKS. Global Optimization Toolbox: Particle Swarm. Natick, MA: The MathWorks, Inc. Documentação oficial. Disponível em: <https://www.mathworks.com/help/gads/particle-swarm.html>. Acesso em: 29 jul. 2026.

RUSSELL, S. J.; NORVIG, P. Inteligência Artificial: Uma Abordagem Moderna. 3. ed. Rio de Janeiro: Elsevier, 2013.